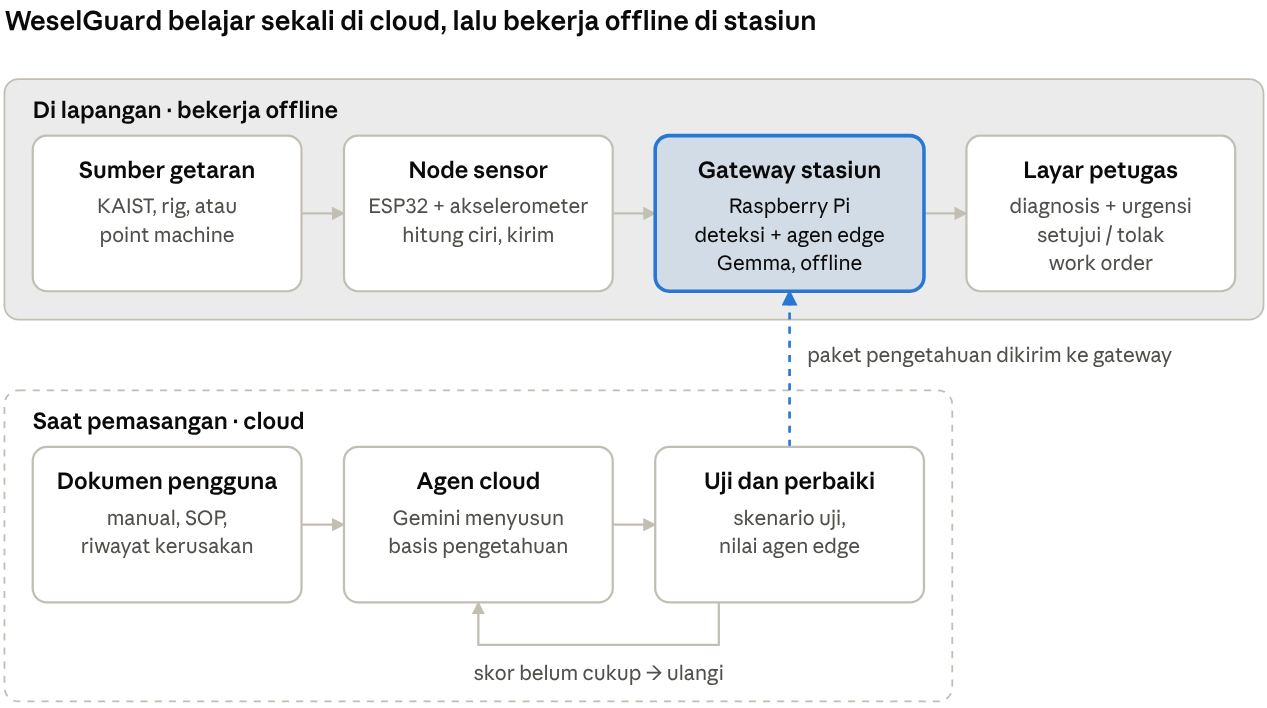
Konsep Kerja WeselGuard

Oct 21, 2026 · @Muhammad Farhan Imanudin

WeselGuard adalah agen perawatan prediktif untuk point machine: sinyal getaran diubah menjadi diagnosis, tingkat urgensi, dan work order dengan rujukan SOP, langsung di stasiun meski tanpa internet, dan petugas tetap memegang keputusan akhir.

Arsitektur

Sistem punya dua sisi: cloud dipakai sekali saat pemasangan atau saat manual diperbarui, sedangkan pekerjaan sehari-hari berjalan di stasiun tanpa internet.



arsitektur WeselGuard

Bagian bergaris putus hanya aktif saat pemasangan. Setelah paket pengetahuan sampai di gateway, seluruh alur di lapangan berjalan sendiri.

Dua Lapis Kecerdasan

Diagnosis dibagi dua lapis, ditambah petugas sebagai penentu: **kode menghitung, agen menafsirkan, petugas memutuskan.**

Lapis	Sifat	Tugas	Belajar dari
Lapis 1: deteksi	Kode biasa, bukan LLM	Mengubah sinyal menjadi ciri (RMS, kurtosis, puncak frekuensi, durasi siklus), membandingkan dengan profil normal, memberi kandidat kondisi dan indeks kesehatan 0–100	Data getaran berlabel (normal, bantalan aus, poros tidak sejajar)
Lapis 2: agen	LLM (Gemma di gateway)	Menafsirkan hasil lapis 1 dengan manual, mengecek riwayat, menentukan urgensi, menyusun work order atau eskalasi	Manual contoh dan instruksi agen; tidak melihat sinyal mentah dan tidak melihat label
Petugas	Manusia	Menyetujui, menolak, atau menghentikan agen	Pengalaman lapangan

Deteksi saja belum agentic; itu klasifikasi biasa. Yang membuat WeselGuard agentic adalah apa yang terjadi setelah deteksi.

Yang Membuatnya Agentic

Agen menerima tujuan, merencanakan langkah sendiri, memakai tools sesuai kasus, lalu bertindak dengan manusia memegang kendali di titik penting.

Tujuan agen edge: saat ada anomali, tentukan apa yang terjadi, seberapa mendesak, dan apa yang harus dilakukan petugas.

Tool	Fungsi
`deteksi_anomali`	Mengambil hasil lapis 1: ciri, kandidat kondisi, skor kecocokan, indeks kesehatan
`cari_manual`	Mencari subbab manual yang paling relevan dengan gejala (RAG)
`cek_riwayat`	Membaca kejadian dan keputusan petugas sebelumnya pada wesel yang sama

Tool	Fungsi
`nilai_urgensi`	Menentukan Klasifikasi A, B, atau C berdasarkan Bab 4 manual
`buat_work_order`	Menyusun langkah pemeriksaan berdasarkan Bab 5 manual, berstatus menunggu persetujuan
`eskalasi`	Meminta teknisi ahli saat pola tidak dikenal

Agen memilih urutan dan kombinasi tools sendiri. Contohnya, kalau riwayat menunjukkan kejadian serupa yang sudah terbukti, keyakinan naik; kalau hasil lapis 1 dan gejala di manual tidak cocok, agen langsung ke `eskalasi`.

Kendali manusia:

- Agen hanya mengusulkan; work order berlaku setelah disetujui petugas.
- Layar petugas punya tombol untuk menghentikan agen kapan saja.
- Log langkah agen selalu ditampilkan, sehingga petugas dan juri bisa melihat alasan di balik setiap keputusan.

Kondisi gagal: kalau skor kecocokan rendah, atau hasil lapis 1 bertentangan dengan manual, agen tidak menebak. Agen menyatakan pola tidak dikenal dan meminta teknisi ahli, sesuai subbab 3.6 manual.

Pengetahuan Agen dan Self-Improvement

Bahan yang dipakai agen

Bahan	Isinya	Dipakai oleh
Manual contoh	Bab 1–5: komponen, batas operasi (PM 10/2011), diagnosis getaran, urgensi A/B/C dan pemeriksaan (diadaptasi PM 32/2011)	Agen edge, lewat `cari_manual`
Instruksi agen	Peran, tujuan, aturan (selalu sebut sumber, tidak bertindak tanpa persetujuan, eskalasi saat ragu), format output	Agen edge
Definisi tools	Daftar tool dan fungsinya	Agen edge

Bahan	Isinya	Dipakai oleh
Profil normal dan ambang	Angka hasil kalibrasi, disimpan per sumber data	Kode lapis 1, bukan LLM
Riwayat kejadian	Work order yang disetujui atau ditolak	Agen edge, lewat `cek_riwayat`
Skenario uji	Kasus berlabel beserta jawaban benar	**Hanya agen cloud**, sebagai kunci jawaban

Manual tidak diberikan utuh setiap kali agen berpikir. Manual dipotong per subbab dan diindeks (RAG), lalu `cari_manual` hanya mengambil 2–3 potongan yang relevan. Ini menjaga model kecil di gateway tetap cepat dan jawabannya tidak tercampur bagian lain.

Loop self-improvement di cloud

Saat pemasangan atau saat manual diperbarui, agen cloud (Gemini) menyiapkan dan menguji agen edge:

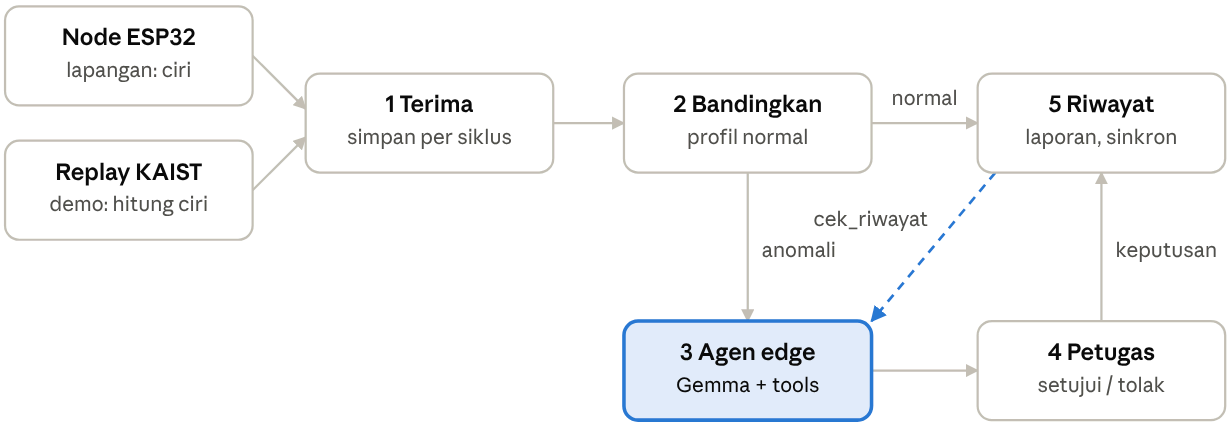
1. Mengindeks dokumen yang diunggah pengguna menjadi basis pengetahuan.
2. Menjalankan agen edge pada skenario uji berlabel.
3. Menilai jawabannya: diagnosis benar, urgensi benar, langkah wajib disebut.
4. Memperbaiki konfigurasi pada kasus yang salah: cara memotong manual, jumlah potongan yang diambil, instruksi agen, dan contoh jawaban.
5. Mengulang sampai skor mencapai ambang, lalu mengirim paket pengetahuan ke gateway.

Skor tiap putaran dicatat apa adanya, termasuk kalau tidak naik. Label skenario uji tidak pernah dikirim ke agen edge.

Alur di Dalam Raspberry Pi

Gateway menjalankan lima tahap untuk setiap siklus perpindahan wesel. Agen hanya dipanggil saat ada anomali, sehingga Pi tidak terbebani menjalankan model untuk setiap siklus normal.

Agen hanya bekerja saat ada anomali; siklus normal langsung dicatat



alur di dalam Raspberry Pi

Di lapangan, ESP32 sudah menghitung ciri getaran, jadi Pi hanya menerima ringkasan. Saat demo, Pi menerima data mentah hasil replay KAIST dan menghitung cirinya sendiri dengan modul yang sama. Setelah ciri tersedia, alurnya identik.

Tahap	Yang dikerjakan	Output
1. Terima dan simpan	Menerima pesan MQTT, menyimpan ciri per wesel per siklus ke database lokal	Riwayat ciri getaran per wesel
2. Bandingkan dengan profil normal	Lapis 1: skor anomali, kandidat kondisi, indeks kesehatan 0–100, dan trennya	Status Normal, Waspada, atau Bahaya
3. Agen edge	Dipanggil hanya saat status bukan Normal atau indeks terus menurun; memakai tools	Catatan diagnosis dan draf work order, atau eskalasi
4. Kendali petugas	Petugas menyetujui, menolak, atau menghentikan agen	Work order final, keputusan tercatat
5. Riwayat, laporan, sinkron	Menyimpan kejadian untuk `cek_riwayat`, menyusun ringkasan harian, sinkron ke cloud saat online	Riwayat kejadian, laporan harian, tren indeks kesehatan

Output akhir yang dilihat orang: status tiap wesel, catatan diagnosis dengan rujukan manual dan Klasifikasi A/B/C, work order, eskalasi, serta laporan dan tren indeks kesehatan.

Sumber Data dan Format Pesan

Sumber	Isi	Dipakai untuk
Dataset getaran motor KAIST (CC BY 4.0)	Rekaman 120 detik per kondisi pada 25.600 Hz: normal, kerusakan lintasan luar bantalan (3 tingkat), poros tidak sejajar (3 tingkat). Dipotong menjadi siklus 3–5 detik mengikuti batas pembalikan wesel di PM 10/2011	Kalibrasi lapis 1, skenario uji, demo utama
Rig peniru siklus point machine	Motor kecil dengan akselerometer; node ESP32 menghitung ciri dan mengirimnya lewat LoRa ke gateway	Demo langsung di depan juri
Node ESP32 di point machine (lapangan)	Ciri getaran per siklus	Pemasangan setelah hackathon

Semua sumber dikirim dengan format MQTT yang sama, sehingga kode di gateway tidak perlu tahu asal datanya. Tiga aturan:

1. **Label kondisi tidak ikut dikirim ke agen.** Label hanya ada di skenario uji sebagai kunci jawaban.
2. **Profil normal dan ambang disimpan per sumber**, karena satuan dan amplitudo tiap sensor berbeda.
3. **Hasil uji dicatat per sumber**, tidak dicampur.

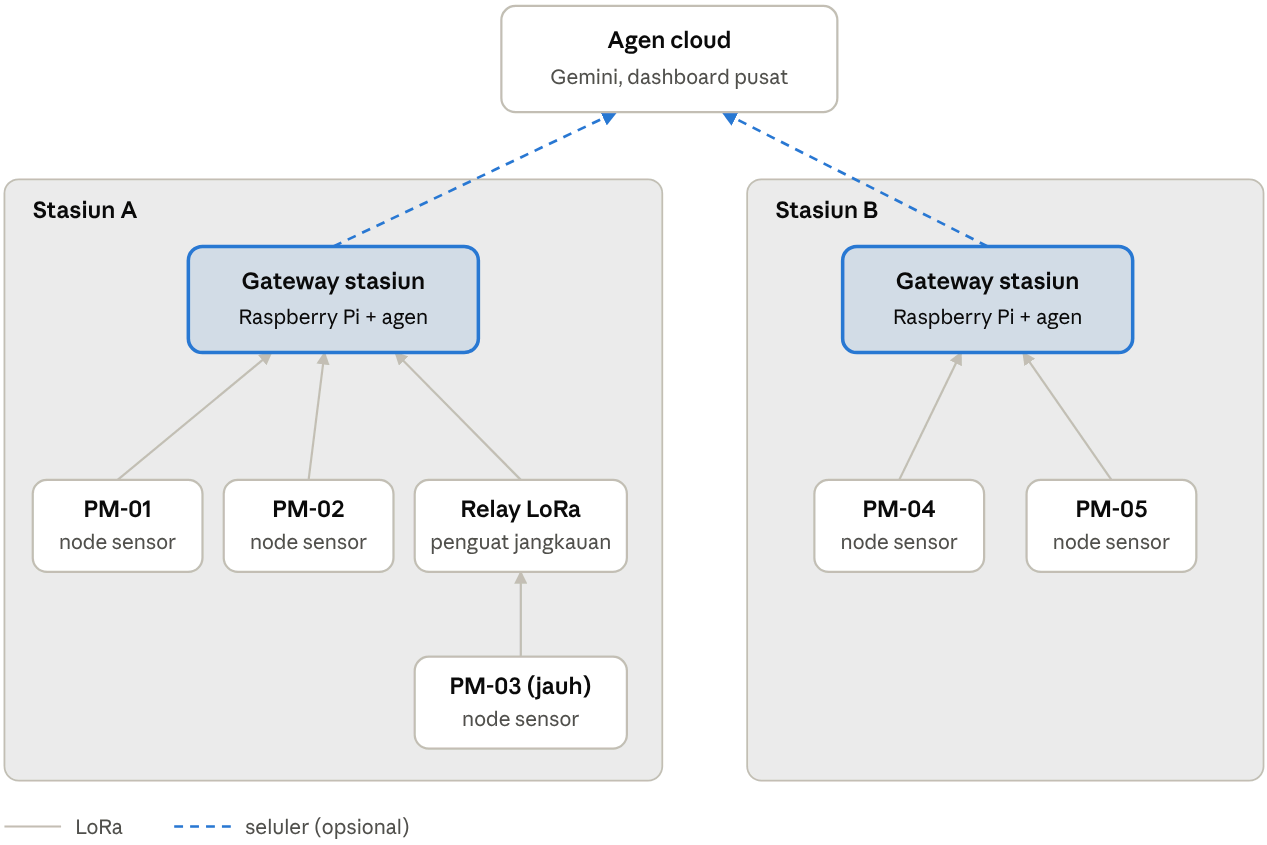
Rincian dataset dan format pesan ada di dokumen Rencana Hackathon WeselGuard, bagian Sumber Data.

Pemasangan Lapangan

Setelah hackathon, setiap point machine dipasangi node sensor yang mengirim ringkasan lewat LoRa ke satu gateway per stasiun. Atas saran mentor, demo hackathon membangun jalur ini untuk satu point machine tiruan: node ESP32 di rig mengirim ciri lewat LoRa ke ESP32 penerima yang tersambung ke Raspberry Pi. Relay, banyak stasiun, dan cloud tetap rencana pengembangan.

Prinsip utama: WeselGuard terpisah total dari sistem persinyalan. Tidak ada data atau daya yang diambil dari kabel atau rangkaian point machine, sehingga statusnya tetap pemantau non-vital.

Node di tiap wesel mengirim ringkasan lewat LoRa ke gateway stasiun



topologi pemasangan lapangan

Bagian	Isi	Tugas
Node di point machine	ESP32, akselerometer, modul LoRa, baterai dengan panel surya	Menghitung ciri getaran per siklus dan mengirim ringkasan beberapa puluh byte

Bagian	Isi	Tugas
Relay LoRa	Node penerus tanpa sensor	Menjangkau wesel yang terlalu jauh dari gateway
Gateway di stasiun	Raspberry Pi dengan penerima LoRa dan agen edge	Meneruskan pesan LoRa ke MQTT lokal, lalu menjalankan lima tahap di atas
Cloud (opsional)	Agen cloud dan dashboard pusat, lewat koneksi seluler	Pembaruan pengetahuan dan pemantauan lintas stasiun; sistem tetap jalan kalau koneksi putus

Ciri dihitung di node karena data getaran mentah (25.600 sampel per detik) terlalu besar untuk LoRa. Gateway meneruskan pesan LoRa ke format MQTT yang sama dengan demo, jadi kode agen tidak berubah.

Yang perlu dicek sebelum pemasangan nyata: pita frekuensi dan aturan perangkat radio di Indonesia, izin pemasangan di area jalur, serta ketahanan perangkat terhadap debu, air, dan getaran kereta yang lewat.

Batasan yang Disampaikan Terbuka

- **Data demo bukan dari point machine sungguhan.** Dataset KAIST adalah getaran motor yang disusun mengikuti siklus point machine; rig adalah peniru. Data lapangan adalah langkah pilot.
- **"Prediktif" berarti deteksi dini.** Sistem mendeteksi kerusakan di tingkat keparahan awal dan menilai urgensinya. Prediksi sisa umur butuh data degradasi jangka panjang; grafik sisa umur di demo adalah ilustrasi dengan jarak waktu yang diasumsikan.
- **Manual adalah manual contoh**, disusun tim dengan merujuk PM 10/2011, PM 32/2011, dan literatur analisis getaran, bukan dokumen resmi operator.
- **WeselGuard hanya memantau.** Sistem tidak mengendalikan wesel dan tidak terhubung ke rangkaian interlocking; setiap tindakan disetujui petugas.